

MEMORIA TRABAJO DISCRETA

Grupo: GP23A

María Caballo Calderón

Lucas García Cucala

1) Organización del trabajo

El programa se distribuye en dos archivos principales: **modular.py**, la librería que contiene los algoritmos, mientras que **imatlab.py** se encarga de leer los comandos, comprobar los argumentos y mostrar los resultados.

No hemos utilizado librerías auxiliares.

2) Diseño general de IMAT-LAB

entrada -> lectura de argumentos -> comando_... -> modular.py -> resultado

Las funciones **leer_enteros** y **leer_sistema** convierten el texto introducido por el usuario en enteros o listas. Después, **ejecutar_comando** decide qué operación ejecutar. Cada función **comando_...** llama a la función correspondiente de **modular.py** y cambia el resultado a texto.

Se han usado dos mensajes para los errores. **NE** significa que el resultado no existe. **NOP** se utiliza cuando el comando está mal escrito.

No hemos añadido algoritmos extra aparte de los que pedía la práctica.

3) Algoritmos de modular.py

- **es_primo:** Para comprobar si un número es primo, primero comprobamos los casos sencillos (los números menores que 2 y algunos divisores pequeños). Después usamos Miller-Rabin. Si alguna de esas comprobaciones falla, sabemos que el número es compuesto. Hacemos divisiones pequeñas antes para ahorrarnos cálculos innecesarios.
- **lista_primos:** Recorremos el intervalo pedido y vamos comprobando qué números son primos con **es_primo**. Vemos el caso del 2 y después probamos números impares (ya que cualquier primo mayor que 2 tiene que ser impar).
- **factorizar:** Para factorizar un número trabajamos con su valor absoluto. Primero sacamos todas las veces que aparecen los factores 2 y 3 y después probamos posibles divisores de la forma $6k \pm 1$. Guardamos en un diccionario cada primo y el número de veces que aparece. Esto se basa en que todo entero mayor que 1 tiene una factorización prima única (Teorema Fundamental de la Aritmética), y saltarnos múltiplos de 2 y 3 reduce bastante el número de pruebas. Busca solo hasta raíz de n .
- **bezout:** Usamos el algoritmo de Euclides extendido para obtener el máximo común divisor y los coeficientes de Bézout, que cumplen $d = ax + by$.
- **mcd:** Empleamos el algoritmo de Euclides clásico por divisiones sucesivas (empleado en clase) y vamos calculando $a \% b$ actualizando las variables hasta que el resto es 0, ya que $\text{mcd}(a, b) = \text{mcd}(b, r)$ (Lema de Euclides). En **mcd_n**, vamos calculando el mcd de los números de la lista de dos en dos de manera sucesiva hasta terminar.
- **coprimos:** Para saber si dos números son coprimos, comprobamos si su MCD es 1.

- **potencia_mod_p:** En vez de multiplicar la base una y otra vez, usamos exponenciación binaria. Vamos dividiendo el exponente entre 2, elevando la base al cuadrado y tomando el módulo en cada paso. De esta forma se necesitan muchas menos multiplicaciones. Así mismo para una mayor eficiencia realizamos las operaciones del bucle directamente a nivel de bits con operadores binarios: usamos “**exp & 1**” para comprobar si el exponente es impar, y “**exp >> 1**” para dividir el exponente entre 2.
- **inversa_mod_p:** La inversa de n módulo p solo existe cuando n y p son coprimos. Por eso calculamos $\text{Bézout}(n,p)$: si el MCD es 1, el coeficiente que acompaña a n nos da una inversa y lo reducimos módulo p . Si el MCD no es 1, la inversa no existe (Teorema de Bézout).
- **euler:** Primero factorizamos n y usamos únicamente sus factores primos distintos con los que aplicamos la fórmula de Euler $\phi(n) = n \cdot \prod (1 - 1/p)$. Así podemos calcular el resultado a partir de la factorización, sin tener que comprobar uno por uno todos los números menores que n .
- **legendre:** Cuando p es un primo impar, usamos el criterio de Euler y calculamos $n^{(p-1)/2}$ módulo p . Si el resultado es 1 devolvemos 1, si p divide a n devolvemos 0 y, en el resto de casos, devolvemos -1. Para hacer esta potencia empleamos nuestro algoritmo **potencia_mod_p**.
- **resolver_sistema_congruencias:** Cada ecuación del tipo $a \cdot x = b \pmod{p}$ se transforma primero en una congruencia más sencilla (siempre que tenga solución). Después vamos combinando las congruencias de dos en dos. Para saber si una ecuación lineal se puede resolver usamos la condición de divisibilidad del MCD y, para juntar las soluciones, aplicamos el Teorema Chino del Resto, asumiendo que los módulos son coprimos dos a dos.

4) Conclusiones

- La práctica nos ha servido para aplicar en Python los contenidos vistos en Matemática Discreta y reutilizar algoritmos básicos, como Euclides y Bézout, en problemas más complejos, así como conocer otros tantos algoritmos nuevos como Miller-Rabin.

5) Bibliografía

[Rincón Matemático](#), Teorema de Miller-Rabin

Transparencias tema 2 matemática discreta

[CP Algorithms ES](#), para hacer potencia_mod_p

[Universidad Autónoma de Madrid](#), Legendre